Operasi Aritmatika

NRP	: 1. 3223600044
	2. 3223600055
	3. 3223600058
Nama	: 1. Devano Auriza Khairindra Heriyanto
	2. Arimbi Kusuma Jati
	3. Ihsanta Zaki Sanjaya
Kelas	: 2 Teknik Komputer B
Matakuliah	: Praktikum Organisasi Mesin dan Bahasa Assembly
Tanggal	: Kamis, 29 Agustus 2024
Dosen	: Dr. Setiawardhana ST, MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA 2024

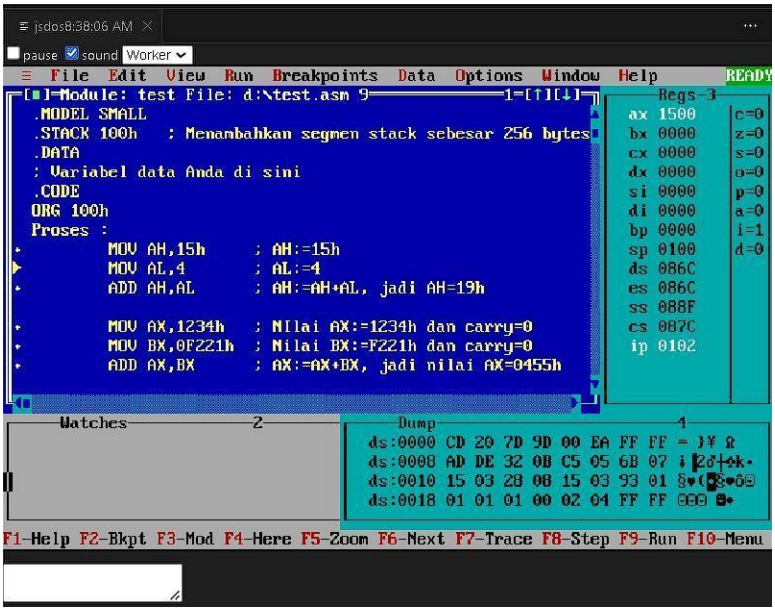
OPERASI ARITMATIKA

PROGRAM PENAMBAHAN DAN DEBUG

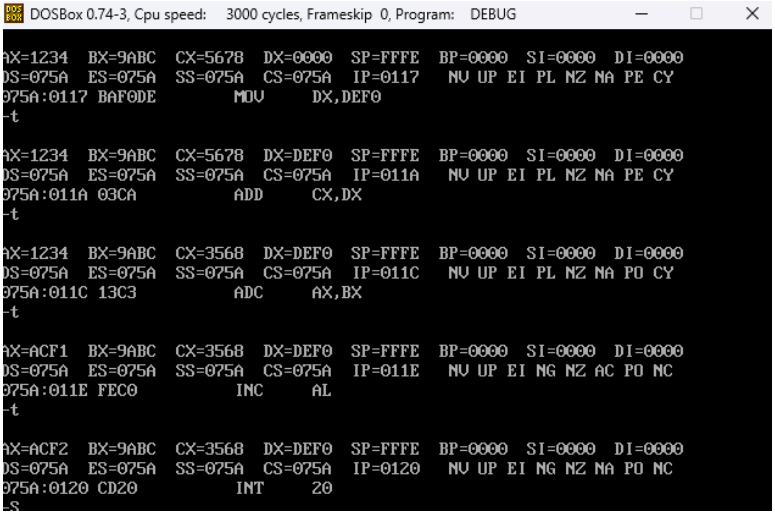
Berikut merupakan sintax dari program penambahan dan debug yang berfungsi untuk melihat penambahan yang dilakukan oleh berbagai perintah.

```
.MODEL SMALL
.STACK 100h   ; Menambahkan segmen stack sebesar 256 bytes
.DATA
; Variabel data Anda di sini
.CODE
ORG 100h
Proses :
        MOV AH,15h      ; AH:=15h
        MOV AL,4        ; AL:=4
        ADD AH,AL       ; AH:=AH+AL, jadi AH=19h

        MOV AX,1234h    ; NIlai AX:=1234h dan carry=0
        MOV BX,0F221h ; Nilai BX:=F221h dan carry=0 ADD
        AX,BX           ; AX:=AX+BX, jadi nilai AX=0455h

        MOV AX,1234h    ; AX = 1234h CF = 0
        MOV BX,9ABCh    ; BX = 9ABCh  CF = 0
        MOV CX,5678h    ; BX = 5678h  CF = 0
        MOV DX,0DEF0h   ; DX = DEF0h  CF = 0
        ADD CX,DX       ; CX = 3568h  CF = 1
        ADC AX,BX       ; AX = AX+BX+CF = ACF1

        INC AL          ; AL:=AL+1, nilai pada AL ditambah 1

        INT 20h
END     Proses
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 1 Hasil debug program pada vs code



Gambar 2 Hasil debug program penambahan pada DOSBox

Analisa pada program operasi penambahan ini, terdapat serangkaian operasi penambahan pada register dalam arsitektur x86. Program dimulai dengan mendefinisikan segment stack sebesar 256 bytes, sehingga menyediakan ruang sementara untuk data yang dibutuhkan selama eksekusi. Dengan menggunakan ADD dan ADC:

- AH diberi nilai ‘15h’ dan AL diberi ‘4’. Kedua register dijumlahkan dengan ‘ADD AH, AL’, sehingga menghasilkan AH yang bernilai ‘19h’.
- AX diberi nilai ‘1234h’ dan BX diberi ‘0F221h’. Saat ‘ADD AX,BX’ dieksekusi, menghasilkan AX yang bernilai ‘0455h’
- CX diberi nilai ‘5678h’ dan DX diberi ‘0DEF0h’. Penjumlahan keduanya menghasilkan CX yang bernilai ‘3568h’ dengan carry flag yang di set menjadi 1. Selanjutnya ‘ADC AX, BX’ digunakan untuk menambahkan AX, BX, dan carry flag, sehingga menghasilkan AX yang bernilai ‘ACF1h’

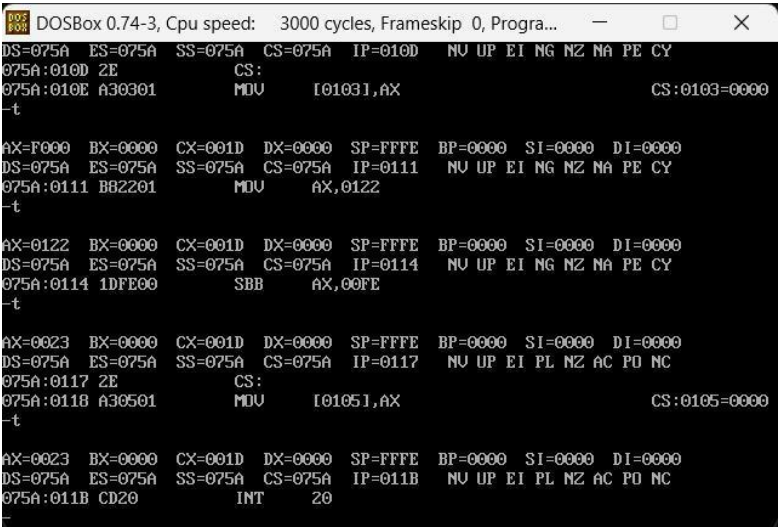
Increment AL juga digunakan untuk menambahkan 1 pada AL, sebagai ilustrasi bagaimana increment bekerja pada assembly. Kemudian ditutup dengan interrupt 20h untuk keluar dari program. Beberapa instruksi ini dapat memanipulasi nilai register secara langsung, dengan memanfaatkan operasi bitwise dan carry flag untuk menghasilkan hasil akhirPerubahan pada register jugu disebabkan penjumlahan dan status flag yang dipengaruhi oleh operasi tersebut.

PROGRAM OPERASI PENGURANGAN

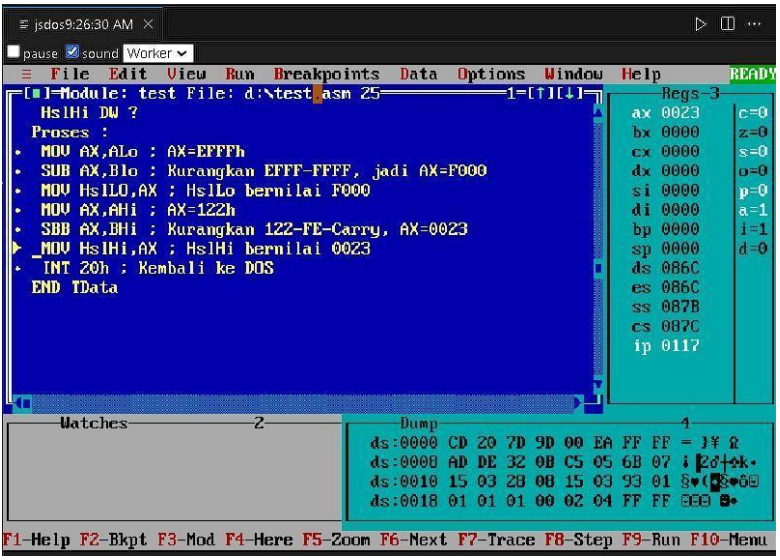
Berikut merupakan sintax dari program operasi pengurangan yang berfungsi untuk mengurangkan angka 122EFFF-0FEFFFF.

```
   .MODEL SMALL
  .CODE
 ORG 100h
TData :
  JMP Proses ; Lompat ke Proses
 ALo EQU 0EFFFh
  AHi EQU 122h
 BLo EQU 0FFFFh
 Bhi EQU 0FEh
 HslLo DW ?
 HslHi DW ?
Proses :
  MOV AX,ALo ; AX=EFFFh
  SUB AX,Blo ; Kurangkan EFFF-FFFF, jadi AX=F000
 MOV HslLO,AX ; HslLo bernilai F000
  MOV AX,AHi ; AX=122h
  SBB AX,BHi ; Kurangkan 122-FE-Carry, AX=0023 MOV
 HslHi,AX ; HslHi bernilai 0023
 INT 20h ; Kembali ke DOS END
TData
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 3 Hasil debug program pengurangan pada DOSBox



Gambar 4 Hasil debug program pengurangan pada VS Code

Pada program pengurangan, terdapat operasi aritmatika yang mengurangkan dua bilangan 16-bit. Program ini menggunakan intstruksi ‘SUB’ dan ‘SBB’ yang digunakan untuk operasi pengurangan dan pengurangan dengan borrow (pinjaman), sehinggda dapat berdampak langsung pada hasil yang disimpan pada register dan memori. SBB memperhitungkan carry flag dari operasi sebelumnya, sehingga menghasilkan output yang sesuai.

- AX diberi nilai ‘EFFh’ (ALo), lalu dikurangi dengan ‘FFFFh’ (BLo) menggunakan instruksi ‘SUB’, sehingga menghasilkan nilai ‘F000h’ pada ‘HslLo’.
- AX diberi nilai ‘122h’ (AHi), lalu dikurangi dengan ‘FE’ (BHi) menggunakan intruksi ‘SBB’, sehingga menghasilkan nilai akhir AX adalah ‘0023h’ pada ‘HslHi’.

Perbedaan antara SUB dan SBB, adalah di mana SBB mengurangi dua nilai dengan memperhitungkan carry flag dari operasi sebelumnya.

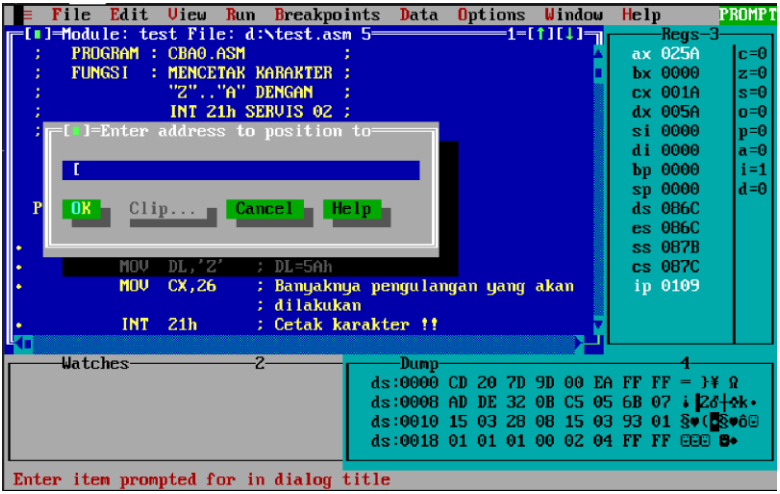
PROGRAM OPERASI DECREMENT

Berikut merupakan sintax dari program operasi pengurangan yang dikhususkan dengan 1. Instruksi ini juga hanya menggunakan 1 byte memory

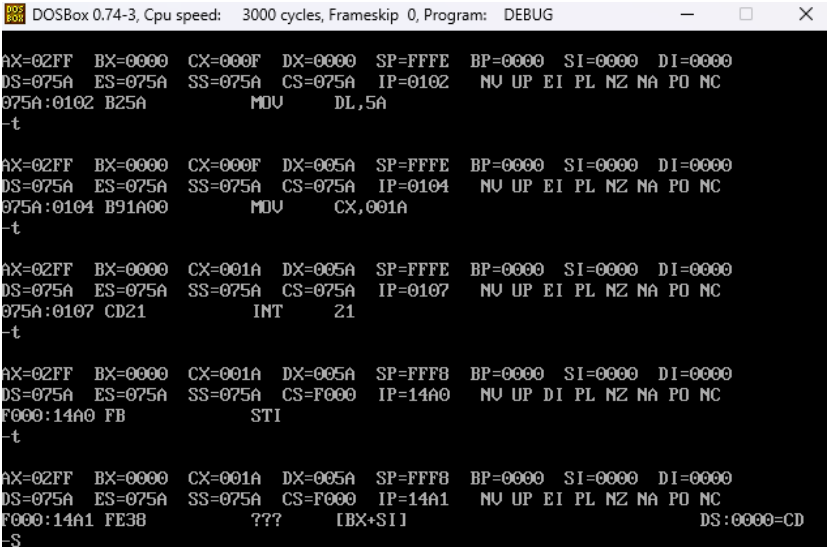
```
;   PROGRAM : CBA0.ASM          ;
;   FUNGSI  : MENCETAK KARAKTER ;
;             "Z".."A" DENGAN   ;
;             INT 21h SERVIS 02 ;
;==========================S’to=;
         .MODEL SMALL
```

```
        .CODE
        ORG 100h
Proses :
  Ulang:
        MOV  AH,02h    ; Nilai servis
        MOV  DL,'Z'    ; DL=5Ah
        MOV  CX,26     ; Banyaknya pengulangan yang akan
                       ; dilakukan
        INT  21h       ; Cetak karakter !!
        DEC  DL        ; Kurang DL dengan 1
        LOOP Ulang     ; Lompat ke Ulang
        INT  20h
END     Proses
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 5 Hasil debug program decrement pada VS Code



Gambar 6 Hasil debug program decrement pada DosBox

Pada program operasi decrement, berfungsi untuk mencetak karakter dari 'Z' hingga 'A' dengan menggunakan interrupt 21h servis 02h. Program ini dimulai dengan pengaturan nilai servis `02h` ke register `AH`, yang merupakan fungsi untuk mencetak karakter ke layar. Register `DL` diatur dengan nilai ASCII 'Z' (5Ah) dan register `CX` diatur dengan jumlah pengulangan yang diperlukan, yaitu 26 kali.

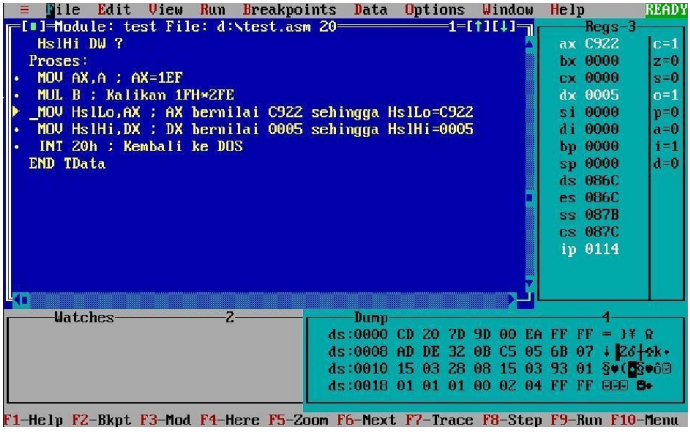
Program kemudian memasuki loop yang mencetak karakter yang tertera di `DL` dan mengurangi nilai `DL` satu per satu dengan instruksi `DEC`, hingga mencapai 'A'. Instruksi `LOOP` digunakan untuk mengulangi proses cetak sebanyak nilai `CX`. Setelah semua karakter dicetak, program berakhir dengan `INT 20h`, yang merupakan interrupt untuk keluar dari program. Program ini memanfaatkan mekanisme decrement untuk mengubah karakter yang dicetak secara bertahap dari 'Z' ke 'A', menunjukkan bagaimana manipulasi nilai register dan penggunaan loop dapat digunakan untuk mengulang tindakan dalam assembly.

PROGRAM OPERASI PERKALIAN

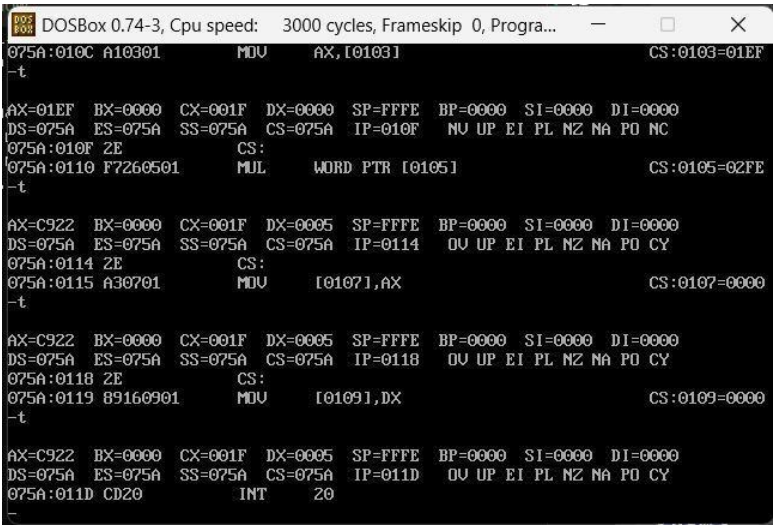
Berikut merupakan sintax dari program operasi perkalian yang berfungsi untuk mengalikan bilangan 16 bit dengan AX dan DX adalah hasilnya.

```
   .MODEL SMALL
  .CODE
 ORG 100h
TData :
  JMP Proses ; Lompat ke Proses
  A DW 01EFh
  B DW 02FEh
 HslLo  DW  ?
HslHi  DW  ?
Proses:
   MOV AX,A ; AX=1EF
  MUL B ; Kalikan 1FH*2FE
  MOV HslLo,AX ; AX bernilai C922 sehingga HslLo=C922
 MOV HslHi,DX ; DX bernilai 0005 sehingga HslHi=0005 INT
 20h ; Kembali ke DOS
END TData
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 7 Hasil debug program perkalian pada VS Code



Gambar 8 Hasil debug program perkalian pada DOS Box

Pada program operasi perkalian, diawali dengan deklarasi data A yang bernilai 01EFh, B bernilai 02FEh, serta HslLo dan HslHi yang nilainya belum diketahui. Definisi variabel ‘A’ dan ‘B’ menggunakan DW dan tidak menggunakan EQU sehingga variabel ‘A’ dan ‘B’ dapat dirubah sesuai keinginan. Setelah program membaca deklarasi ini program akan langsung menjalankan perintah dibaris proses karena sebelumnya telah diberi perintah JMP Proses. Pada bagian proses yang perlu dilakukan adalah memberi nilai pada register AX dengan menggunakan perintah MOV AX, A sehingga saat ini register AX telah bernilai A yaitu 1EF. Setelah nilai register AX dimasukkan maka perintah perkalian dapat dijalankan. Operasi perkalian dapat dijalankan dengan perintah MUL. Syntax perintah MUL adalah:

```
MUL Sumber
```

Pada program yang berjalan terdapat perintah MUL B yang berarti program akan mengalikan nilai B dengan nilai pada register AX. Sehingga komputasi yang akan berjalan adalah perkalian 1EF*2FE atau AX*B. Karena operasi perkalian ini mengalikan nilai sumber yang merupakan 16 bit maka terdapat dua hasil yaitu Word rendah dan Word tinggi.

Register AX digunakan untuk menyimpan Word rendah dan DX

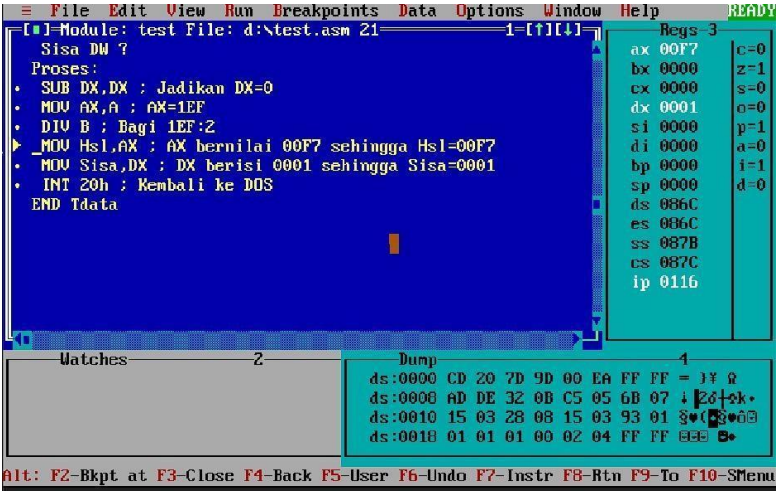
akan menyimpan Word tinggi. Hasil rendah dari operasi ini adalah C922 dan hasil tingginya adalah 0005. Kemudian untuk menampilkan kedua hasil tersebut maka perintah MOV dijalankan dengan syntax MOV HslLo, AX untuk mengcopy hasil rendah dan dimasukkan ke dalam register AX dan perintah MOV HslHi, DX untuk mengcopy hasil tinggi dan dimasukkan ke dalam register DX.

PROGRAM OPERASI PEMBAGIAN

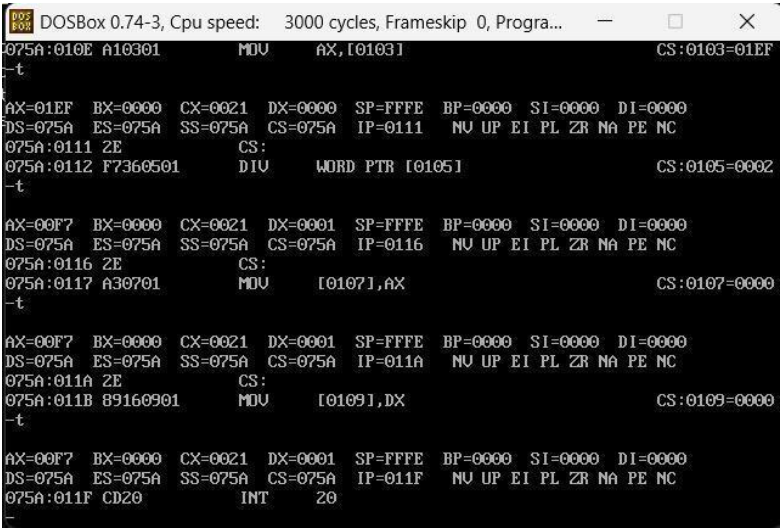
Berikut merupakan sintax dari program operasi pembagian yang berfungsi untuk membagi bilangan 16 bit dengan AX dan DX sebagai hasilnya.

```
.MODEL SMALL
  .CODE
 ORG 100h
TData :
  JMP Proses ; Lompat ke Proses
  A DW 01EFh
   B DW 2
 Hsl DW ?
Sisa DW ?
Proses:
  SUB DX,DX ; Jadikan DX=0
 MOV AX,A ; AX=1EF
  DIV B ; Bagi 1EF:2
  MOV Hsl,AX ; AX bernilai 00F7 sehingga Hsl=00F7
 MOV Sisa,DX ; DX berisi 0001 sehingga Sisa=0001
 INT 20h ; Kembali ke DOS
END Tdata
```

Apabila program dengan sintax di atas di-debug maka akan menampilkan hasil seperti di bawah ini.



Gambar 9 Hasil debug program pembagian pada VS Code



Gambar 10 Hasil debug program pembagian dengan DOS Box

Pada operasi pembagian digunakan perintah DIV. Perintah DIV pada syntax operasi pembagian berfungsi sebagai perintah untuk mengambil nilai yang akan dibagi. Secara umum penggunaan syntax

untuk pembagian adalah:

```
DIV Sumber
```

Dalam praktikum ini syntax yang digunakan adalah DIV B, yang berarti program akan membagi dengan nilai B yang telah dideklarasikan bernilai 2 sebelumnya. Perlu diingat bahwa pada operasi pembagian register yang akan digunakan adalah register AX sehingga nilai pada register AX akan dibagi dengan B sesuai perintah pada syntax DIV. Nilai yang dideklarasi pada syntax ini adalah ‘A’ dengan nilai 01EFh. Kemudian, deklarasi nilai B bernilai 2 dan nilai untuk deklarasi ‘Hsl’ dan ‘Sisa’ belum diketahui. Selanjutnya program akan menjalankan perintah JMP Proses yang akan membuat program lompat ke baris proses. Pada baris proses terdapat perintah SUB yang membagi register DX dengan DX sehingga nilai DX menjadi 0.

Selanjutnya perintah MOV AX, A berarti nilai A telah dimasukkan pada register AX sehingga kini register AX memiliki nilai yaitu 1EF, sekadar informasi angka 0 pada AX tidak dibaca karena letak 0 di depan angka tidak berarti. Proses operasi pembagian akhirnya dijalankan dengan perintah DIV B. Sehingga saat register AX dibagi dengan B maka program membagi nilai pada data AX:B atau 1EF:2. Karena sumber merupakan operand 16 bit maka hasil dari pembagian yang bernilai 00F7 akan disimpan pada AX dan sisa pembagian 0001 akan disimpan register DX. Untuk menampilkan nilai yang tersimpan dan mengubah nilai awal pada AX dan DX maka digunakan perintah ‘MOV’ yang akan mengcopy nilai ‘Hsl’ pada AX dan ‘Sisa’ pada DX.

KESIMPULAN

Pada praktikum ini, kami berhasil mengimplementasikan berbagai operasi aritmatika dasar menggunakan instruksi assembler seperti `ADD`, `ADC`, `SUB`, `SBB`, `MUL`, dan `DIV`.

1. Operasi penjumlahan dilakukan dengan instruksi `ADD` dan `ADC`, di mana perbedaan antara penjumlahan biasa dan penjumlahan dengan carry flag dijelaskan secara mendetail.
2. Operasi pengurangan memanfaatkan instruksi `SUB` dan `SBB` untuk mengurangi bilangan besar dengan mempertimbangkan "borrow".
3. Operasi pengurangan dengan pinjaman (decrement) menggunakan perintah DEC yang mana kebalikan dari perintah INC. DEC digunakan bila akan mengurangi tujuan yang berupa register maupun memory dengan 1.
4. Operasi perkalian menunjukkan bagaimana dua bilangan 16-bit dikalikan menggunakan `MUL`, menghasilkan output 32-bit yang terbagi antara register `DX` dan `AX`.
5. Operasi pembagian menggunakan instruksi `DIV` untuk membagi bilangan 16-bit dan menyimpan hasil serta sisanya di register `AX` dan `DX`.

Praktikum ini secara keseluruhan memberikan pemahaman yang jelas tentang bagaimana operasi aritmatika dasar dilakukan di tingkat mesin dengan penggunaan instruksi dan register prosesor secara langsung.